

RAG-LLM Chatbot

Workflow Documentation

Endpoint-level workflows for every feature — FastAPI · PostgreSQL · pgvector

1. Overview

This document expands the RAG-LLM Chatbot feature specification into endpoint-level workflow documentation. For every feature it lists the HTTP method and path (where the feature is a callable endpoint), the request shape, the step-by-step backend workflow, the response shape, and the error cases a client should handle. Features with no endpoint entry are internal steps triggered as part of another call — this is noted explicitly so the document doubles as an implementation map, not just a feature list.

The audio/video transcription pipeline has been removed from this version, per current project scope.

2. Authentication & Account Management

2.1 Sign Up

POST /api/v1/auth/signup

Registers a new user account with a hashed password.

Request

- username: string
- email: string
- password: string

Workflow

1. Client submits username, email, and password.
2. Server checks the case-insensitive unique indexes on users.email and users.username for collisions.
3. Password is hashed with bcrypt (never stored in plaintext) and inserted into the users table.
4. Server issues an access token and refresh token immediately, or requires email verification first, depending on product requirements.

Response

201 Created — returns access_token, refresh_token, and the created user profile (id, username, email).

Error handling

- 409 Conflict — username or email already registered.
- 422 Unprocessable Entity — password does not meet minimum strength requirements.

Tables involved: users, user_sessions

2.2 Login

POST /api/v1/auth/login

Authenticates an existing user and starts a session.

Request

- email: string
- password: string

Workflow

5. Client submits email and password.
6. Server looks up the user by lowercased email and verifies the password against password_hash.
7. On success, server issues a short-lived JWT access token and a long-lived refresh token.
8. Refresh token is stored in user_sessions along with device and ip_address for session management.

Response

200 OK — returns access_token, refresh_token, token_type.

Error handling

- 401 Unauthorized — email not found or password does not match. Use the same generic error message for both cases to avoid revealing which emails are registered.

Tables involved: *users, user_sessions*

2.3 Token Refresh

POST /api/v1/auth/refresh

Exchanges a valid refresh token for a new access token without re-entering credentials.

Request

- refresh_token: string

Workflow

9. Client sends its refresh token once the access token has expired.
10. Server checks user_sessions: the token exists, is_revoked is false, and expires_at has not passed.
11. Server issues a new access token (and optionally rotates the refresh token for additional security).

Response

200 OK — returns a new access_token (and new refresh_token if rotation is enabled).

Error handling

- 401 Unauthorized — refresh token not found, expired, or revoked. Client should force the user back to login.

Tables involved: *user_sessions*

2.4 Logout

POST /api/v1/auth/logout

Ends a session, either on one device or everywhere.

Request

- refresh_token: string
- all_devices: boolean (optional, default false)

Workflow

12. Client sends its current refresh token to the logout endpoint.
13. Server sets is_revoked = true on that specific user_sessions row.
14. If all_devices is true, server instead sets is_revoked = true on every session belonging to that user_id.

Response

204 No Content.

Error handling

- 401 Unauthorized — refresh token invalid or already revoked; treat as already logged out.

Tables involved: *user_sessions*

2.5 Password Reset / Account Recovery

POST /api/v1/auth/password-reset/request

Lets a user regain access without knowing their current password. This is split into two calls: a request step and a confirm step.

Request

- email: string

Workflow

15. User submits their email address to request a reset.
16. Server looks up the user by lowercased email.
17. Regardless of whether the email exists, the server returns the same generic success response — this is deliberate, see note below.
18. If the email does exist, the server generates a short-lived, single-use reset token, stores only its hash (never the raw token) in a `password_reset_tokens` table, and emails the raw token/link to the user.
19. User follows the emailed link and submits a new password along with the token via POST `/api/v1/auth/password-reset/confirm`.
20. Server hashes the submitted token and looks it up; verifies it is unexpired and unused.
21. Server updates `password_hash`, marks the reset token as used, and revokes all existing `user_sessions` rows for that user, forcing re-login everywhere in case the account was compromised.

Response

200 OK on the request step (generic message every time). 200 OK on the confirm step if the token was valid.

Error handling

- The request step never returns 404 for an unknown email — that would let an attacker enumerate registered accounts by testing addresses one at a time.
- 400 Bad Request on confirm — token invalid, expired, or already used.

Tables involved: `users`, `password_reset_tokens` (new table), `user_sessions`

3. Conversations

3.1 Create Conversation

POST `/api/v1/conversations`

Starts a new chat thread for a user.

Request

- `model_name`: string
- `system_prompt`: string (optional)
- `generation_config`: object (optional)

Workflow

22. User initiates a new chat, optionally choosing a model.
23. Server inserts a conversations row with a default title, the chosen `model_name`, and an empty `generation_config`.
24. Title remains generic until the first exchange completes.

Response

201 Created — returns the new conversation object (`id`, `title`, `model_name`, `created_at`).

Tables involved: `conversations`

3.2 Auto-Generate Conversation Title

Internal step triggered automatically after the first exchange, not a client-facing endpoint.

Workflow

25. After the first user message and assistant reply are stored, both are sent to the LLM with a short summarization prompt.
26. The returned short phrase updates `conversations.title`.
27. This is a non-blocking enhancement — conversations remain fully usable with a default title if this step is skipped or delayed.

Response

No direct response — reflected the next time the client fetches the conversation.

Tables involved: *conversations, messages*

3.3 List / Archive / Delete Conversations

GET / PATCH / DELETE `/api/v1/conversations{/id}`

Standard lifecycle management for a user's chat history.

Request

- List: `page`, `page_size`, `is_archived` (query params)
- Archive: `is_archived`: boolean (body)

Workflow

28. List: query conversations filtered by `user_id`, ordered by `updated_at`, paginated.
29. Archive: flip `is_archived` to true and exclude archived conversations from the default list view.
30. Delete: removing a conversations row cascades automatically to its messages, attachments, and `conversation_documents` links via foreign key constraints, so no manual cleanup step is required.

Response

200 OK (list/archive) — 204 No Content (delete).

Error handling

- 404 Not Found — conversation does not exist or does not belong to the requesting user.

Tables involved: *conversations, messages, attachments, conversation_documents*

3.4 Full-Text Search Across Conversations

GET `/api/v1/conversations/search`

Lets a user find a past conversation by content rather than scrolling.

Request

- `q`: string (query param)

Workflow

31. A `tsvector` column is maintained on `messages.content` with a GIN index.
32. Search queries run Postgres full-text search against that index, scoped to the requesting user's conversations.
33. Results link back to the specific conversation and message for direct navigation.

Response

200 OK — list of matching messages with `conversation_id`, `snippet`, and `rank`.

Tables involved: *messages*

3.5 Export a Conversation

GET `/api/v1/conversations/{id}/export`

Downloads a conversation as a portable file.

Request

- `format`: string (query param, `markdown` | `pdf`)

Workflow

34. User requests an export of a specific conversation.
35. Server fetches all messages for that `conversation_id` in chronological order.

36. Messages are formatted into Markdown (or converted further to PDF) and returned as a downloadable file.

Response

200 OK — file stream with appropriate Content-Type and Content-Disposition headers.

Error handling

- 404 Not Found — conversation does not exist or does not belong to the requesting user.

Tables involved: *conversations, messages*

4. Messaging

4.1 Send Message + Streamed LLM Response

POST /api/v1/conversations/{id}/messages

The core chat loop.

Request

- content: string
- parent_message_id: uuid (optional)

Workflow

37. Client posts new message content to a conversation.
38. Server inserts a messages row with role = user.
39. Server fetches recent conversation history, truncated to fit the model's context window based on token count rather than row count.
40. If the conversation has scoped documents, RAG retrieval runs (see Section 6) and relevant chunks are injected into the prompt.
41. Server calls the LLM with the system prompt, history, and any retrieved context, streaming the response back to the client over SSE or WebSocket.
42. On completion, server inserts a messages row with role = assistant, recording model_name, token_count, and generation_time.
43. If retrieval was used, corresponding message_citations rows are inserted for every chunk actually included in the prompt.

Response

200 OK — text/event-stream of incremental tokens, followed by a final event containing the persisted message id and citations.

Error handling

- 429 Too Many Requests — rate limit exceeded (see Section 7.1).
- 502 Bad Gateway — upstream LLM call failed; user message remains persisted so nothing is lost.

Tables involved: *messages, conversations, message_citations*

4.2 Chat History Retrieval / Context Building

Internal step run as part of every message send, not a standalone endpoint.

Workflow

44. Query messages for the conversation, ordered by created_at.
45. Apply a token-budget cutoff — older messages are dropped or summarized once the context window limit is approached.
46. Assembled history is combined with any retrieved RAG context and the system prompt before being sent to the model.

Response

Feeds directly into the LLM call in 4.1 — no separate client-facing response.

Tables involved: *messages, conversations*

4.3 Message Edit / Regenerate (Branching)

POST `/api/v1/messages/{id}/regenerate`

Allows editing an earlier message or regenerating a response without destroying history.

Request

- `content`: string (only for edit; omitted for a plain regenerate)

Workflow

47. User edits an earlier message, or requests regeneration of an assistant response.
48. A new messages row is inserted with `parent_message_id` pointing at the message being replaced, rather than overwriting the original row.
49. The client can reconstruct either branch by walking `parent_message_id` links, enabling a conversation-tree view even if the current UI only shows the latest branch.

Response

200 OK — same streamed shape as 4.1, since regeneration re-runs the LLM call.

Error handling

- 404 Not Found — target message does not exist in this conversation.

Tables involved: *messages*

4.4 Feedback (Thumbs Up/Down + Text)

POST `/api/v1/messages/{id}/feedback`

Captures qualitative signal on assistant responses.

Request

- `is_helpful`: boolean
- `feedback_text`: string (optional)

Workflow

50. User rates an assistant message; server updates `is_helpful` (true, false, or left null if unrated).
51. Optional free-text feedback is stored in `feedback_text`.

Response

204 No Content.

Tables involved: *messages*

5. File & Document Handling

5.1 File Upload

POST `/api/v1/documents`

Accepts PDF, DOCX, image, and other files, either attached to a message or scoped to a conversation.

Request

- `file`: multipart/form-data
- `conversation_id`: uuid (optional, for conversation-level scoping)

Workflow

52. Client uploads a file, either inline with a message or as a conversation-level document.
53. Server computes a SHA-256 content_hash of the file before any storage write.
54. If a document with that hash already exists, the upload is linked rather than re-stored (see 5.4).
55. Otherwise, the file is saved to disk or object storage and a new documents row is inserted with status = uploading.

Response

202 Accepted — returns the document id and status; processing continues asynchronously.

Error handling

- 413 Payload Too Large / 415 Unsupported Media Type — rejected before storage, see 5.2.

Tables involved: *documents, attachments, user_documents, conversation_documents*

5.2 Upload Validation

Middleware applied to every upload before it reaches storage, not a separate endpoint.

Workflow

56. Enforce a maximum file size before accepting the upload body.
57. Sniff actual file bytes to confirm the MIME type rather than trusting the client-supplied value.
58. Reject any file type outside the supported allowlist (PDF, DOCX, PNG, JPEG, and similar).

Response

Rejected uploads never reach 5.1's storage step — the request fails fast with a 4xx.

Error handling

- 413 Payload Too Large
- 415 Unsupported Media Type

Tables involved: *documents, attachments*

5.3 Document Processing Pipeline

GET /api/v1/documents/{id}/status

Turns an uploaded file into searchable, embedded text. Runs as a background worker; the endpoint listed here is for polling status.

Workflow

59. A background worker picks up documents with status = processing.
60. Text is extracted with a format-appropriate library (PDF via pypdf, DOCX via python-docx, images via OCR if applicable).
61. Extracted text is split into chunks and inserted into document_chunks.
62. Each chunk is embedded using the system's pinned embedding model, and the resulting vector is inserted into chunk_embeddings.
63. documents.status is updated to ready, or failed with a logged reason if any step errors out.
64. The client polls this endpoint, or receives a push notification, when status changes, so the UI can reflect progress.

Response

200 OK — returns status (uploading | processing | ready | failed) and, if failed, a reason.

Tables involved: *documents, document_chunks, chunk_embeddings, embedding_models*

5.4 Global Document Deduplication

Internal logic inside the upload step (5.1), not a separate endpoint.

Workflow

65. Before storing a new file, its `content_hash` is checked against the unique index on `documents.content_hash`.
66. If a match exists, the existing `documents` row is reused: a `user_documents` row links the uploader if not already linked, and a `conversation_documents` row scopes it into the current conversation.
67. No duplicate storage, extraction, or embedding work occurs for content that already exists in the system.

Response

Reflected in 5.1's response — a deduped upload returns `status = ready` immediately instead of uploading.

Tables involved: `documents`, `user_documents`, `conversation_documents`

5.5 Remove a Document from a Conversation

DELETE `/api/v1/conversations/{conv_id}/documents/{doc_id}`

Unscopes a document from one conversation without necessarily deleting it system-wide.

Workflow

68. Server deletes the specific `conversation_documents` row linking that document to that conversation.
69. The underlying `documents`, `document_chunks`, and `chunk_embeddings` rows remain intact for reuse elsewhere.
70. A separate cleanup job periodically removes `documents` rows with no remaining `user_documents` or `conversation_documents` references, rather than deleting on every unlink.

Response

204 No Content.

Error handling

- 404 Not Found — document is not currently scoped to this conversation.

Tables involved: `conversation_documents`, `documents`

6. Retrieval-Augmented Generation

6.1 Query Embedding + Similarity Search

Internal step triggered inside 4.1 whenever a conversation has scoped documents.

Workflow

71. The user's query is embedded using the same pinned embedding model used at ingestion time.
72. A similarity search runs across `chunk_embeddings`, joined through `document_chunks` and scoped to the documents linked to the current conversation via `conversation_documents`.
73. Results are ordered by cosine distance using the HNSW index, and the top-k chunks are returned.

Response

Feeds directly into prompt construction — not directly exposed to the client.

Tables involved: `chunk_embeddings`, `document_chunks`, `conversation_documents`, `embedding_models`

6.2 Re-ranking (Optional)

Internal step, run immediately after 6.1 when enabled.

Workflow

74. The top-k chunks from vector search are passed to a cross-encoder re-ranking model.
75. Chunks are re-scored for relevance to the specific query and re-ordered.
76. A smaller final subset is selected for inclusion in the prompt, trading a small latency cost for better answer grounding.

Response

Feeds directly into prompt construction — not directly exposed to the client.

Tables involved: *document_chunks* (in-memory re-ranking, no schema change)

6.3 Citation Tracking

Internal step, runs at the end of 4.1 once the LLM response is complete. Citations are surfaced to the client as part of the message response.

Workflow

77. After the LLM generates a response using retrieved context, the chunks actually included in the prompt are identified.
78. A `message_citations` row is inserted per chunk, recording the `chunk_id`, `embedding_model_id`, `similarity_score`, and `rank`.
79. The composite foreign key into `chunk_embeddings` guarantees a citation can never reference a chunk/model pair that was never actually embedded.

Response

Returned as a citations array on the final streamed message event in 4.1.

Tables involved: *message_citations, chunk_embeddings, document_chunks, messages*

6.4 Prompt Injection Safeguards

Applied inside prompt construction, before every LLM call that includes retrieved content.

Workflow

80. Retrieved chunks are wrapped in the prompt template with an explicit label identifying them as untrusted reference material, not instructions.
81. The system prompt reinforces that instructions only come from the system role, not from retrieved content.
82. This is a mitigation, not a complete defense — retrieved content is still passed to the model and should be treated as an ongoing risk area.

Response

No direct response — a prompt-construction safeguard, not a callable endpoint.

Tables involved: *document_chunks* (prompt-construction logic, no schema change)

7. Platform & Operations

7.1 Rate Limiting / Abuse Prevention

Middleware applied to the message-send endpoint (4.1), not a standalone endpoint.

Workflow

83. Each request to the message-send endpoint is checked against a per-user rate limit (token-bucket or sliding-window) backed by Redis.
84. Requests exceeding the limit are rejected before reaching the LLM call, avoiding unnecessary cost and load.

Response

429 Too Many Requests when the limit is exceeded, with a Retry-After header.

Tables involved: *Redis* (no PostgreSQL table required)

7.2 Usage & Token Quota Tracking

GET /api/v1/users/me/usage

Tracks consumption for cost control and potential future billing.

Workflow

85. token_count is already recorded per message at generation time.
86. A background rollup aggregates token usage per user per day or month, stored in Redis counters or a dedicated summary table.
87. Quota checks can gate the message-send endpoint once a user's usage crosses a configured threshold.

Response

200 OK — returns current period usage and remaining quota.

Error handling

- 403 Forbidden — returned from 4.1 if the user's quota is exhausted.

Tables involved: *messages, usage_summary (new table, optional)*

7.3 Health & Readiness Endpoints

GET /health, /ready

Standard operational endpoints for deployment and monitoring.

Workflow

88. /health confirms the API process itself is running.
89. /ready confirms the process can successfully reach PostgreSQL, Redis, and the LLM backend before being marked available to receive traffic.

Response

200 OK when healthy/ready; 503 Service Unavailable otherwise.

Tables involved: *None (infrastructure-level check)*

7.4 Test Coverage

Guards the two areas where subtle bugs are most costly. Not an endpoint — a testing requirement.

Workflow

90. Integration tests cover the full auth flow: sign up, login, refresh, logout, and revocation behavior.
91. Integration tests cover RAG retrieval: correct scoping to a conversation's documents, correct similarity ordering, and correct citation recording.

Response

N/A

Tables involved: *All auth and RAG-related tables*